

10- پیوست شماره 2: کدهای پیش پردازش داده ها به زبان پایتون

simple_data.py

```

import json
import random
import pandas as pd
import numpy as np
from pathlib import Path
from sklearn.preprocessing import MinMaxScaler

def find_label(record):
    label_keys = ['label', 'Label', 'class', 'Class', 'target', 'Target',
                  'avclass', 'labels', 'classes']

    for key in label_keys:
        if key in record:
            value = record[key]
            if isinstance(value, dict):
                if 'label' in value:
                    value = value['label']
                elif 'name' in value:
                    value = value['name']

            try:
                return int(value)
            except (ValueError, TypeError):
                if isinstance(value, str):
                    if value.lower() in ['benign', 'good', 'safe']:
                        return 0
                    else:
                        return 1

    return None

def find_features(record):
    feature_keys = ['features', 'feature', 'data', 'vector']

    for key in feature_keys:
        if key in record and isinstance(record[key], list) and len(record[key]) > 0:
            return [float(x) for x in record[key]]

    numbers = []

```

```
for key, value in record.items():
    if key.lower() in ['label', 'class', 'target', 'avclass', 'labels',
'classes']:
        continue

    if isinstance(value, (int, float)):
        numbers.append(float(value))
    elif isinstance(value, list) and all(isinstance(x, (int, float)) for x in
value):
        numbers.extend([float(x) for x in value])

return numbers if numbers else None

def is_good_record(record):
    return find_features(record) is not None and find_label(record) is not None

def fix_vector_length(vec, target_length):
    if len(vec) < target_length:
        return vec + [0.0] * (target_length - len(vec))
    return vec[:target_length]

def extract_balanced_samples(input_file, output_file, samples_per_class=2500,
random_seed=42):
    random.seed(random_seed)

    class_zero = []
    class_one = []

    print(f"Reading from {input_file}...")

    with open(input_file, 'r', encoding='utf-8') as f:
        for line in f:
            line = line.strip()
            if not line:
                continue

            try:
                record = json.loads(line)

                if not is_good_record(record):
                    continue
```

```

        label = find_label(record)

        if label == 0 and len(class_zero) < samples_per_class:
            class_zero.append(record)
        elif label == 1 and len(class_one) < samples_per_class:
            class_one.append(record)

        if len(class_zero) >= samples_per_class and len(class_one) >=
samples_per_class:
            print(f"Reached target at row")
            break

    except json.JSONDecodeError:
        continue

    print(f"\nSummary:")
    print(f"  Class 0: {len(class_zero)} samples")
    print(f"  Class 1: {len(class_one)} samples")

    if len(class_zero) < samples_per_class:
        print(f"Warning: Only {len(class_zero)} samples for class 0 (needed
{samples_per_class})")

    if len(class_one) < samples_per_class:
        print(f"Warning: Only {len(class_one)} samples for class 1 (needed
{samples_per_class})")

    all_samples = class_zero + class_one
    random.shuffle(all_samples)

    print(f"\nSaving {len(all_samples)} records to {output_file}...")

    with open(output_file, 'w', encoding='utf-8') as f:
        for record in all_samples:
            f.write(json.dumps(record, ensure_ascii=False) + '\n')

    print(f"\nExtraction completed!")
    print(f"  Total: {len(all_samples)} records")
    print(f"  Class 0: {len(class_zero)}")
    print(f"  Class 1: {len(class_one)}")
    print(f"  Output: {output_file}")

    return all_samples

```

```
def convert_jsonl_to_dataframe(jsonl_file, max_records=5000):
    records = []
    labels = []
    feature_length = None

    print(f"\nReading {jsonl_file}...")

    with open(jsonl_file, 'r', encoding='utf-8') as f:
        for line_num, line in enumerate(f, 1):
            if len(records) >= max_records:
                break

            line = line.strip()
            if not line:
                continue

            try:
                record = json.loads(line)

                if not is_good_record(record):
                    continue

                features = find_features(record)
                label = find_label(record)

                if features is None or label is None:
                    continue

                if feature_length is None:
                    feature_length = len(features)
                    print(f"Feature vector length: {feature_length}")

                features = fix_vector_length(features, feature_length)
                records.append(features)
                labels.append(label)

                if line_num % 1000 == 0:
                    print(f"  Processed {line_num} lines, collected {len(records)} records")

            except json.JSONDecodeError:
                continue

    if not records:
        print("No valid records found!")
```

```

        return None

    print(f"\nData collection complete:")
    print(f"   Valid records: {len(records)}")
    print(f"   Feature length: {feature_length}")
    print(f"   Class 0: {labels.count(0)}")
    print(f"   Class 1: {labels.count(1)}")

    sample_features = records[0]
    non_zero_count = sum(1 for x in sample_features if x != 0)
    print(f"   Non-zero features in first record:
{non_zero_count}/{feature_length}")

    if non_zero_count == 0:
        print("\nWarning: All features are zero! Check your input file
structure.")

    column_names = [f"feature_{i:04d}" for i in range(feature_length)]
    column_names.append("label")

    data_rows = []
    for i in range(len(records)):
        row = records[i] + [labels[i]]
        data_rows.append(row)

    df = pd.DataFrame(data_rows, columns=column_names)

    feature_cols = [col for col in column_names if col != 'label']

    if non_zero_count > 0:
        print(f"\nNormalizing features...")
        scaler = MinMaxScaler()
        df[feature_cols] = scaler.fit_transform(df[feature_cols])
        print("Normalization complete")
    else:
        print("\nSkipping normalization (all features are zero)")

    return df, feature_length

def save_to_csv(df, output_path):
    df.to_csv(output_path, index=False)
    file_size = Path(output_path).stat().st_size / (1024 * 1024)
    print(f"\nCSV saved to: {output_path}")
    print(f"File size: {file_size:.2f} MB")

```

```
    return output_path

def run_pipeline(input_file="test_features.jsonl",
                 balanced_file="balanced_dataset.jsonl",
                 csv_output="dataset.csv",
                 samples_per_class=2500):

    print("=" * 60)
    print("DATA PROCESSING PIPELINE")
    print("=" * 60)

    if not Path(input_file).exists():
        print(f"Error: {input_file} not found!")
        return None

    extract_balanced_samples(
        input_file=input_file,
        output_file=balanced_file,
        samples_per_class=samples_per_class
    )

    df, feature_count = convert_jsonl_to_dataframe(balanced_file,
max_records=samples_per_class * 2)

    if df is None:
        return None

    save_to_csv(df, csv_output)

    print("\n" + "=" * 60)
    print("SAMPLE OF FIRST 3 RECORDS (first 10 features):")
    print("=" * 60)

    feature_cols = [col for col in df.columns if col != 'label']
    display_cols = feature_cols[:10] + ['label']
    print(df[display_cols].head(3))

    print("\n" + "=" * 60)
    print("PIPELINE COMPLETED")
    print("=" * 60)
    print(f"\nOutput file: {csv_output}")
    print(f"Total records: {len(df)}")
    print(f"Total features: {len(df.columns) - 1}")
    print(f"Label column: 'label'")
```

```
    return df

if __name__ == "__main__":
    result = run_pipeline(
        input_file="test_features.jsonl",
        balanced_file="balanced_5000_samples.jsonl",
        csv_output="normalized_dataset.csv",
        samples_per_class=2500
    )

    if result is not None:
        print(f"\nFinal verification:")
        print(f"  Records: {len(result)}")
        print(f"  Class 0: {(result['label'] == 0).sum()}")
        print(f"  Class 1: {(result['label'] == 1).sum()}")
```